

# XPAC Customer Dataset — Capacity & Resource Simulation

Local benchmark from 4,000 to 200,000 customer records, in two layers: the **database** (write throughput, CPU, RAM, storage, query latency) and the **running application** — the project's own Laravel API driven over HTTP against the same data. Every dimension carries a mark. Run 21 September 2026 against a local MariaDB instance and a local copy of the schema; no production system was touched.

| DB ROWS TESTED                                          | SQL QUERIES TIMED                 | APP ROWS TESTED                                                | API REQUESTS TIMED                                 |
|---------------------------------------------------------|-----------------------------------|----------------------------------------------------------------|----------------------------------------------------|
| <b>589,000</b>                                          | <b>245</b>                        | <b>1,040,108</b>                                               | <b>65</b>                                          |
| 200,000 seeded + 389,000 migrated, over 6 dataset sizes | 7 query shapes × 5 runs × 7 sizes | 200k customers plus billing accounts, payments and status rows | 3 endpoints × 5 sizes against the real Laravel app |

## Scorecard — is it good or bad?

| Dimension                                   | Measured at 200,000 records                                      | Mark  |
|---------------------------------------------|------------------------------------------------------------------|-------|
| Server RAM headroom (database work)         | 91.9 MB total (mysqld + PHP) = 0.6% of 16 GB                     | GOOD  |
| PHP request memory (application work)       | 528.3 MB peak against a 512 MB limit — limit reached             | FAIL  |
| Storage growth                              | 659 bytes/record, linear — 125.7 MB for 200k                     | GOOD  |
| Bulk insert throughput (direct)             | 18,605 rows/s, flat from empty to 200k                           | GOOD  |
| Bulk migration throughput (OFFSET paging)   | 2,134 rows/s at 200k, down from 11,854 rows/s at 10k             | BAD   |
| SQL: first page of the customer list        | 0.3 ms — rides the primary key                                   | GOOD  |
| SQL: single lookup by account_no            | 310 ms unindexed (0.30 ms with an index)                         | BAD   |
| SQL: COUNT(*) and GROUP BY aggregates       | 195 ms and 310 ms — full scans                                   | BAD   |
| SQL: deep pagination (90% offset)           | 229 ms — cost grows with offset depth                            | WATCH |
| API: one customer — GET /customers/{id}     | 247 ms end to end (227 ms of it framework boot)                  | GOOD  |
| API: customer list — GET /customers         | HTTP 500 — out of memory at 50,000 records and above             | FAIL  |
| App screen: customer table (paged /billing) | page 1 in 11.1 s; times out from page 50 of 200 — never finishes | FAIL  |
| Same screen, on a server-class DB cache     | 2.3 s per page, no timeouts — 200 pages is still ~8 min          | BAD   |

**How the marks are assigned.** Single SQL query: GOOD ≤ 50 ms, WATCH ≤ 250 ms, BAD above. API request (what a user waits for): GOOD ≤ 300 ms, WATCH ≤ 1 s, BAD ≤ 3 s, FAIL beyond that or on an HTTP error. Bulk write: GOOD ≥ 10,000 rows/s, WATCH ≥ 3,000, BAD below. RAM: GOOD under 25% of the host, WATCH under 60%, BAD above. Storage: GOOD under 1 KB per record. Thresholds are stated up front so a mark can be argued with; every number behind them is in the tables that follow.

## Verdict

**The hardware and the database hold up at 200,000 records. The application does not.** On capacity there is nothing to worry about: the dataset occupies **125.7 MB** on disk, the PHP worker never exceeded **12.0 MB**, mysqld peaked at **79.9 MB** — together **0.6%** of the 16 GB host — and raw insert speed held flat at **18,605 rows/s** from an empty table to 200k. What fails is software. **The customer table screen never finishes loading at 200,000 records.** It loads through /api/billing in pages of 1,000 and keeps going until it holds the whole table — 200 requests; page 1 takes 11.1 s, page 10 takes 25.5 s, and from about page 50 every request dies on the 60-second execution limit. Separately, the unpaginated GET /api/customers endpoint returns **HTTP 500 from 50,000 records upward**, out of memory against PHP's 512 MB limit. Fetching one customer, by contrast, stays flat at about **247 ms** at every size — so nothing is wrong with the machine or the schema's ability to serve a row. Reading is the other half: with no secondary indexes on the table, a single lookup by account\_no costs **310 ms** at 200k rows against 0.30 ms once indexed, and a bulk migration through OFFSET paging drops from 11,854 to **2,134 rows/s**. None of this needs a bigger server; it needs pagination, indexes and keyset paging.

## Test environment

| Component | Configuration                                      |
|-----------|----------------------------------------------------|
| CPU       | AMD Ryzen 5 4500 — 6 cores / 12 threads @ 3.60 GHz |
| RAM       | 16 GB (17,063,915,520 bytes)                       |
| OS        | Windows 10 Pro build 19045                         |

| Component           | Configuration                                                                            |
|---------------------|------------------------------------------------------------------------------------------|
| Database            | MariaDB 10.4.32 (XAMPP, local, 127.0.0.1)                                                |
| Runtime             | PHP 8.2.12 CLI, memory_limit 512M                                                        |
| InnoDB buffer pool  | 16.0 MB (XAMPP default — 8× smaller than the 200k dataset; see findings)                 |
| Durability / packet | innodb_flush_log_at_trx_commit=1, max_allowed_packet=1.0 MB                              |
| Table under test    | customers — 28 data columns + PK, InnoDB / utf8mb4 (as shipped by the Laravel migration) |
| Write path          | batch 500 rows, one multi-row prepared INSERT per transaction (mirrors db_migrator.php)  |

## Write capacity and resource usage by dataset size

| Records        | Wall time (s) | Rows/s | PHP CPU (s) | MySQL CPU (s) | CPU % (1 core) | PHP peak RAM (MB) | mysqld RSS (MB) | Table size (MB) | Bytes /row | Mark  |
|----------------|---------------|--------|-------------|---------------|----------------|-------------------|-----------------|-----------------|------------|-------|
| 4,000 (cold)   | 2.10          | 1,903  | 0.05        | 0.23          | 13%            | 12.0              | 69.8            | 3.5             | 922        | BAD   |
| 10,000         | 0.84          | 11,854 | 0.12        | 0.61          | 87%            | 12.0              | 71.2            | 6.5             | 683        | GOOD  |
| 25,000         | 2.89          | 8,645  | 0.33        | 2.20          | 88%            | 12.0              | 73.6            | 16.5            | 694        | WATCH |
| 50,000         | 9.26          | 5,399  | 0.66        | 6.30          | 75%            | 12.0              | 73.6            | 31.6            | 662        | WATCH |
| 100,000        | 29.65         | 3,373  | 1.33        | 24.19         | 86%            | 12.0              | 73.6            | 63.6            | 667        | WATCH |
| 200,000        | 93.73         | 2,134  | 2.27        | 88.00         | 96%            | 12.0              | 73.9            | 125.7           | 659        | BAD   |
| 200,000 (seed) | 10.75         | 18,605 | 2.59        | 7.39          | 93%            | 6.0               | 62.5            | 125.7           | 659        | GOOD  |

The shaded row is the seed phase — 200,000 rows inserted directly, no source read — and is the reference for pure write capacity. The scale rows above it also read each page back out of the source table with LIMIT/OFFSET, as a real migration does, and that read is where the time goes. The 4,000-row step carries cold-start overhead, so peak migration throughput shows at 10,000 rows rather than the smallest step.

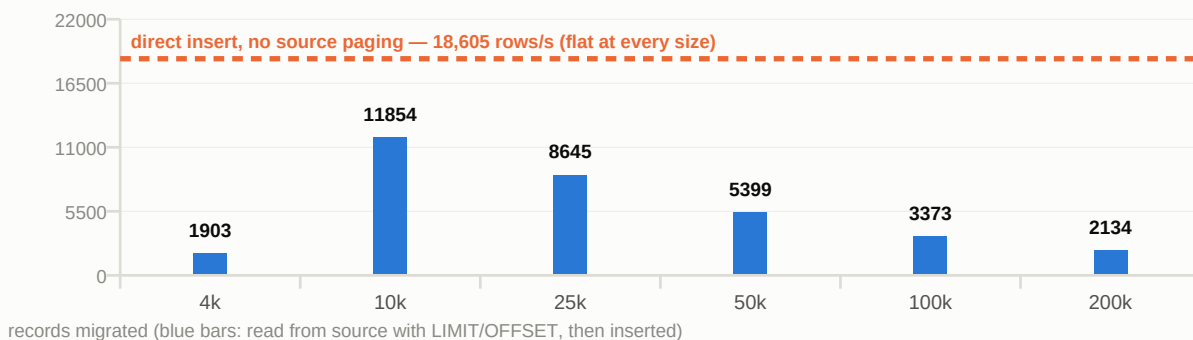
## How this was measured

A 200,000-row source table was generated locally, then each scale step loaded that many rows into a freshly created destination table — batch 500, one multi-row prepared INSERT per transaction, the same strategy db\_migrator.php uses. CPU seconds and resident memory were sampled from Windows process counters for both the PHP worker and mysqld at every step boundary, PHP memory from memory\_get\_peak\_usage(), and table size from information\_schema after ANALYZE TABLE. After each load, 7 query shapes were run 5 times each and the median taken. Finally, four secondary indexes were added at 200,000 rows and the same queries re-run. 589,000 rows were written in total across 176 s of wall time.

**Scope.** Single-user, single-worker measurements on a developer workstation with other software running; no concurrent load was simulated, so these figures describe capacity and cost per operation, not peak multi-user throughput. Data was synthetic but shaped like real records (Philippine names, addresses, plans, storage URLs); table definitions came from the project's own migrations, and the production schema snapshot in db\_schema.json confirms the live customers table has the same 29 columns and, like the one tested, only a primary key. The application layer ran the project's real backend under php artisan serve (PHP built-in dev server, one worker, 512 MB memory limit) against a separate local database, so the absolute response times carry dev-server overhead that php-fpm with OPcache would reduce — the memory ceiling and the shape of the growth would not change. Storage figures are InnoDB estimates. All three test databases are local and can be dropped at any time.

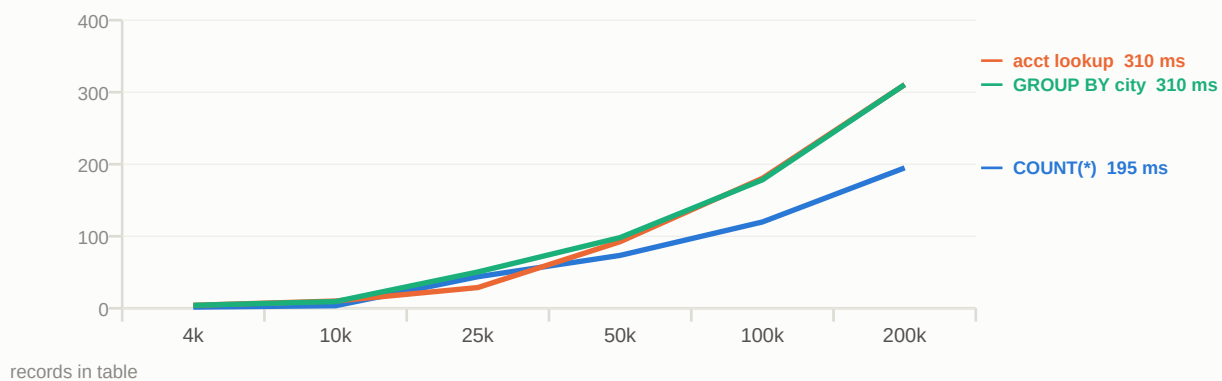
## Migration throughput collapses with OFFSET paging — raw insert speed does not

rows written per second · batch size 500 · single PHP worker



## Unindexed scan latency grows with row count

median of 5 runs, milliseconds · table as shipped (primary key only)



## Query latency by dataset size (median ms)

| Query shape                            | 4k  | 10k  | 25k  | 50k  | 100k  | 200k  | 4k → 200k | Mark  |
|----------------------------------------|-----|------|------|------|-------|-------|-----------|-------|
| COUNT(*) — record total                | 1.6 | 3.6  | 43.8 | 73.4 | 119.8 | 194.9 | 120×      | WATCH |
| First page, 25 rows (ORDER BY id DESC) | 0.3 | 0.3  | 0.3  | 0.3  | 0.2   | 0.3   | 1×        | GOOD  |
| Deep page, 25 rows at 90% offset       | 2.5 | 9.8  | 27.5 | 86.6 | 132.1 | 229.2 | 90×       | WATCH |
| LIKE search on name / account no.      | 1.0 | 1.2  | 0.9  | 0.8  | 0.8   | 0.8   | 1×        | GOOD  |
| Single lookup by account_no            | 4.2 | 10.1 | 28.8 | 92.5 | 180.4 | 310.5 | 75×       | BAD   |
| GROUP BY city, top 10                  | 4.1 | 9.4  | 50.4 | 98.2 | 178.3 | 310.5 | 76×       | BAD   |
| Filter by organization_id, 25 rows     | 0.5 | 0.3  | 0.3  | 0.3  | 0.3   | 0.2   | 1×        | GOOD  |

Only the first-page listing stays flat — it rides the primary key. Every other shape degrades roughly linearly with row count because the table ships with no secondary indexes.

## Effect of adding four secondary indexes at 200,000 rows

| Query shape                            | No index (ms) | With index (ms) | Change         |
|----------------------------------------|---------------|-----------------|----------------|
| COUNT(*) — record total                | 194.9         | 35.1            | 5.6× faster    |
| First page, 25 rows (ORDER BY id DESC) | 0.3           | 0.4             | no real change |
| Deep page, 25 rows at 90% offset       | 229.2         | 245.7           | 1.1× slower    |
| LIKE search on name / account no.      | 0.8           | 0.7             | no real change |
| Single lookup by account_no            | 310.5         | 0.3             | 1,035× faster  |
| GROUP BY city, top 10                  | 310.5         | 50.0            | 6.2× faster    |
| Filter by organization_id, 25 rows     | 0.2           | 0.7             | no real change |

Rows marked *no real change* are sub-millisecond both ways — inside measurement noise; the deep page gets no help because its cost is the discarded OFFSET rows. The four indexes were built in 2.4 s of wall time (2.3 s CPU) and added 23.1 MB, taking the table from 125.7 MB to 148.8 MB (+18%).

## Application-level test — the real API under load

The figures above measure the database. This section measures **the application**: the project's own Laravel backend, running its real controllers, models and Sanctum authentication against a local copy of the schema, driven over HTTP with a real Sanctum token. Each request was issued 3 times per dataset size. The dataset here is richer than the SQL test — every customer also has a billing account, an online-status row, two transactions and, for a quarter of them, a payment-portal log, so the eager-loaded relations and the endpoint's pre-aggregation queries do real work. The API server is restarted before each dataset size, so the peak-memory figure below belongs to that size alone rather than carrying over from the previous one.

**GET /api/customers** — CustomerController::index, unpaginated. Live and reachable, and wrapped by customerService.getCustomers() in the frontend; the customer screen itself uses the paged route measured after this one.

| Records | HTTP | Median response     | Payload | PHP peak RAM (MB) | MySQL CPU (s) | Mark |
|---------|------|---------------------|---------|-------------------|---------------|------|
| 4,000   | 200  | 1,010 ms            | 3.6 MB  | 110.6             | 0.2           | BAD  |
| 25,000  | 200  | 5,637 ms            | 22.3 MB | 413.5             | 1.3           | FAIL |
| 50,000  | 500  | failed after 8.1 s  | —       | 528.3             | 5.3           | FAIL |
| 100,000 | 500  | failed after 12.1 s | —       | 519.0             | 12.9          | FAIL |
| 200,000 | 500  | failed after 10.2 s | —       | 522.2             | 26.2          | FAIL |

The endpoint calls Customer::with([...])->get() — it loads every customer row and its relations into PHP memory, builds an array for each one and returns the lot in a single response. That works at 25,000 records (22.3 MB of JSON) and **stops working between 25,000 and 50,000**: from 50,000 records on, every call died with *Allowed memory size of 512 MB exhausted* after 8–16 seconds of work. No pagination, no limit, no streaming — the cost scales with the table, and the 512 MB PHP limit is reached long before the hardware is.

### Point lookup and framework baseline (median ms)

| Endpoint                                       | 4k  | 25k | 50k | 100k | 200k | Mark |
|------------------------------------------------|-----|-----|-----|------|------|------|
| GET /api/customers/{id} — one customer         | 230 | 256 | 266 | 262  | 247  | GOOD |
| GET /api/user — minimal route (framework boot) | 234 | 244 | 232 | 236  | 227  | GOOD |

These two are flat across the whole range, and that is the useful contrast: fetching one customer costs the same at 200,000 records as at 4,000. Almost all of it is fixed framework boot — the minimal route costs nearly as much as the real one — so on this dev server roughly 234 ms of every response is Laravel starting up, not data. Under php-fpm with OPcache that overhead largely disappears; the list endpoint's failure would not.

### The screen users actually open — GET /api/billing, paged 1,000 at a time

The customer table in the app does not call the endpoint above. frontend/src/store/billingStore.ts asks for /api/billing?page=1&per\_page=1000 and then keeps requesting the next page until the entire table is in browser memory — **200 requests** at 200,000 records. So the page's real cost is the sum of all of them, and each one has to succeed.

| Page requested  | Rows skipped | HTTP | Median response        | Mark |
|-----------------|--------------|------|------------------------|------|
| Page 1          | 0            | 200  | 11.1 s                 | FAIL |
| Page 10         | 9,000        | 200  | 25.5 s                 | FAIL |
| Page 50         | 49,000       | 500  | timed out (60 s limit) | FAIL |
| Page 100 (half) | 99,000       | 500  | timed out (60 s limit) | FAIL |
| Page 200 (last) | 199,000      | 500  | timed out (60 s limit) | FAIL |

At 200,000 records this screen cannot finish loading. Page 1 takes **11.1 s**, page 10 takes **25.5 s**, and from around page 50 every request dies on PHP's 60-second execution limit — so the table stalls after roughly 49,000 of 200,000 records. Even if every page had returned at page 1's speed, the loop would need **37 minutes** and about **339 MB** of transfer to finish. Two causes, both in BillingController::index: it re-aggregates the *entire* transactions and payment\_portal\_logs tables into PHP arrays on every page request, and it pages with OFFSET over an unindexed created\_at sort, so each page costs more than the one before.

## Does a bigger server fix it? — sizing against a VPS-4 class host

The candidate host is **8 vCores, 24 GB RAM, 200 GB SSD NVMe, 3 Gbps** at US\$23.37 / month ex. GST. To test what that buys, the same code and the same 200,000-record dataset were re-run with the limits such a server would actually be given — InnoDB buffer pool raised from the 16.0 MB default to 2 GB, PHP memory limit 4 GB — with one variable moved at a time.

| Request (200,000 records)       | 16 MB pool<br>(as tested) | 2 GB pool<br>(server-class) | Mark at 2 GB |
|---------------------------------|---------------------------|-----------------------------|--------------|
| Customer table, page 1 of 200   | 10.4 s                    | 2.3 s                       | <b>BAD</b>   |
| Customer table, page 50         | 500 after 75 s            | 2.0 s                       | <b>BAD</b>   |
| Customer table, page 200 (last) | 500 after 75 s            | 1.9 s                       | <b>BAD</b>   |

**Memory for the database is the single biggest win available.** Nothing but the pool size changed between those two columns: pages that took 10 s or failed outright now return in about 2 s, and the deep pages that never came back now do. That is the one thing the VPS genuinely fixes — the test machine was starving MySQL of cache, and a 24 GB host would not.

### Resource by resource

| Resource                    | Measured need at 200,000 customers                             | VPS-4 offers                 | Mark         |
|-----------------------------|----------------------------------------------------------------|------------------------------|--------------|
| Disk — relational data      | 297 MB for all five tables, i.e. 1.52 KB per customer          | 200 GB NVMe                  | <b>GOOD</b>  |
| Disk — customer attachments | 6 upload slots per customer: ~230–610 GB if kept on the server | 200 GB NVMe                  | <b>BAD</b>   |
| RAM — database cache        | 297 MB working set; 2 GB pool measured above                   | 24 GB                        | <b>GOOD</b>  |
| RAM — per PHP request       | 821.8 MB peak on the unpaginated endpoint, which still failed  | 24 GB, shared by all workers | <b>WATCH</b> |
| CPU                         | ~1.1–1.5 s of MySQL CPU per page request, single-threaded      | 8 vCores                     | <b>WATCH</b> |
| Bandwidth                   | 339 MB to load the customer table once                         | 3 Gbps, unlimited            | <b>GOOD</b>  |

**Verdict on the host.** The specification is not the problem and 200 GB is generous for the relational data — 1.52 KB per customer means even a million customers is under 2 GB. Two caveats decide whether it is enough. First, **customer attachments must not live on this disk**: six upload slots per customer at typical phone-photo sizes is 230–610 GB at 200,000 customers, which does not fit; the schema stores them as URLs, so keep that pointing at object storage. Second, **vCores are not the same as this test machine's cores** — every request measured here is single-threaded, so eight of them raises how many staff can be served at once, not how fast any one page loads, and a shared vCore is typically slower than the desktop core these timings came from. Expect per-request times at or slightly above the 2 GB-pool column, not below it.

**What the host cannot fix.** Even with a 4 GB PHP limit and a 900-second execution limit, `GET /api/customers` still failed at 200,000 records — after 29 s and 821.8 MB, with *MySQL server has gone away*, because eager-loading relations for every customer builds an `IN ( . . . )` clause of 200,000 ids that exceeds the 1 MB `max_allowed_packet`. And at 2 s per page the customer table still needs 200 pages, about **8 minutes**, to finish loading. Hardware moves those numbers; only the code removes them.

## Key findings

- **The customer table screen cannot load at 200,000 records at all.** This is the one users would notice first. The screen loads through `/api/billing` a thousand rows at a time and keeps going until it has the whole table — **200 requests** at this size. Page 1 takes **11.1 s**, page 10 **25.5 s**, and from about page 50 every request hits the 60-second execution limit and returns 500, so the table stops at roughly **49,000 of 200,000** records. The controller re-aggregates the entire transactions and portal-log tables on every page, then pages by OFFSET over an unindexed sort — so the work grows with both the payment history and the page depth.
- **The unpaginated customers endpoint fails too, and it fails early.** `GET /api/customers` answers in 1.0 s at 4,000 records and 5.6 s at 25,000, then returns **HTTP 500 at 50,000 records and every size above** — PHP runs out of memory after 8–16 seconds of work. The controller loads the entire table plus eager-loaded relations into memory and serialises it in one response (22.3 MB of JSON at 25,000 records), so the ceiling is the 512 MB PHP limit, not the machine. No screen calls it today, but it is authenticated, reachable and already wrapped by a frontend service function — it will break the first thing that uses it.
- **RAM is not the constraint.** The PHP worker held steady at **12.0 MB** peak from 4k to 200k rows — batch-500 streaming means memory does not scale with dataset size. `mysqld`'s resident set moved only from **69.8 MB** to **79.9 MB** across the whole run. Together they use under **0.6%** of the 16 GB host.
- **Storage is linear and predictable.** Roughly **659 bytes per customer record** including the primary key, so **125.7 MB** at 200k rows. A million records would need about **0.6 GB** for this table alone — before indexes, invoices or attachments.
- **CPU is single-threaded and it is the migration's cost centre.** Migrating 200k rows consumed **90.3 s** of CPU over **93.7 s** of wall time — **96%** of one core but only 8% of this 12-thread machine, so 11 of 12 threads sat idle. Note the split: MySQL burned **88.0 s** against PHP's **2.3 s**. The work is in the database reading pages, not in PHP transforming rows.

- **Insert speed is flat; OFFSET paging is what degrades.** This is the run's most actionable result. The seed phase inserted 200k rows into a growing table at a steady **18,605 rows/s** with no decline. The migration passes, which read the same rows back with LIMIT/OFFSET, fell from **11,854 rows/s** at 10k to **2,134 rows/s** at 200k — a **5.6×** slowdown. With batch 500, the last pass re-scanned roughly **40 million rows** to deliver 200k, because every OFFSET discards all preceding rows — a quadratic cost that gets worse, not better, as the customer base grows. The same trap hits the app's own listings: paging to 90% of the table costs **229 ms** at 200k rows against **2.5 ms** at 4k. Keyset pagination removes it on both sides.
- **The real risk at 200k rows is unindexed reads.** A single customer lookup by `account_no` takes **310 ms** at 200k rows versus **4.2 ms** at 4k — a full table scan every time. Adding the four secondary indexes cut it to **0.30 ms**. Any page that searches, filters or counts customers will feel this long before storage or RAM matter.
- **Queries that stop early stay fast at any size.** First-page listing (**0.3 ms** at 200k), the LIKE search (**0.8 ms**) and the organization filter (**0.2 ms**) barely moved between 4k and 200k rows, because each fills its 25-row LIMIT and stops. Indexing is therefore not a blanket win: afterwards the organization filter (0.2 → 0.7 ms, both trivial) and the deep page (229 → 246 ms) measured slightly *slower*, because the index forces a sort the primary-key scan gave away free. Index the point lookups and aggregates that actually hurt, and confirm with EXPLAIN.

## Recommendations

- **Fix the customer table screen first — it is the only thing users cannot work around.** Two changes inside `BillingController::index`: (1) stop re-aggregating the whole transactions and payment\_portal\_logs tables on every request — join or subquery them for the page's 1000 accounts only; (2) page by keyset instead of OFFSET, and index the sort column. Then stop the browser store from looping every page into memory: fetch the page the user is looking at, and let search and filters run server-side. Without these, the screen stops loading at roughly 49,000 records.
- **Paginate GET /api/customers as well.** Same failure, same cause: HTTP 500 from 50,000 records up, and 5.6 s even at 25,000. Replace `Customer::with([...])->get()` with `->paginate(25)` (or a cursor), and move the two pre-aggregation queries over transactions and payment\_portal\_logs to a join or a per-page subquery so they stop scanning the whole payment history on every call. This is a code change in one controller method, not a capacity problem — the same server serves one customer in 247 ms.
- **Switch the migrator's source reader from OFFSET to keyset paging** (`WHERE id > :last_id ORDER BY id LIMIT n` instead of `LIMIT n OFFSET m`). This is the single highest-value change measured here: it should close most of the gap between 2,134 rows/s and the 18,605 rows/s direct-insert baseline. Apply the same fix to customer list pagination in the app.
- **Add secondary indexes on customers.account\_no, organization\_id, last\_name and city** before the table passes ~50k rows. Cost: 23.1 MB of index data, 2.4 s to build at 200k rows; benefit: account\_no lookup 310 ms → 0.30 ms. Index writes slow inserts, so create them *after* a bulk load.
- **Raise innodb\_buffer\_pool\_size** on whichever server hosts this data — it is only **16.0 MB** here while the table alone is **125.7 MB**, so reads cannot stay cached — the likeliest contributor to the read latencies above. 1–2 GB is a reasonable target and trivial against 16 GB of RAM.
- **Keep the migrator's batch size at 500.** PHP memory stayed flat at 12.0 MB regardless of dataset size, so larger batches buy nothing and would approach the 1.0 MB max\_allowed\_packet ceiling on this server.
- **Plan storage at roughly 62.9 MB per 100,000 customer records** (74.4 MB with the four indexes), then re-measure with invoices, payments and attachments included — at real ratios those tables, not customers, will dominate total database size.